

WEEK 3: SELLER AND INVENTORY MANAGEMENT SYSTEM

Scenario

The Organic Grocery Store gets its products from different suppliers. The system needs to store supplier details and keep track of the stock available for each product. This helps the store know which products are available and which products need to be restocked.

1. Create Seller and Inventory Tables

Seller Table

The Seller table stores information about the suppliers who provide products to the store.

```
SQL> CREATE TABLE Seller (seller_id INT PRIMARY KEY, seller_name VARCHAR(50), email VARCHAR(100));  
Table created.
```

Product Table

The Product table stores the products supplied by the sellers.

```
SQL> CREATE TABLE Product (product_id NUMBER PRIMARY KEY, product_name VARCHAR2(100) NOT NULL, seller_id NUMBER, CONSTRAINT fk_product_seller FOREIGN KEY (seller_id) REFERENCES Seller(seller_id));  
Table created.
```

Inventory Table

The Inventory table stores the current stock quantity of each product.

```
SQL> CREATE TABLE Inventory ( inventory_id NUMBER PRIMARY KEY, product_id NUMBER UNIQUE NOT NULL, stock_quantity NUMBER CHECK (stock_quantity >= 0), CONSTRAINT fk_inventory_product FOREIGN KEY (product_id) REFERENCES Product(product_id));  
Table created.
```

2. Establish Relationships

Seller and Product

A seller can supply many products.

Relationship:

Seller → Product = One-to-Many (1:N)

The seller_id connects the seller with the products they supply.

Product and Inventory

Each product has information about its available stock.

Relationship:

Product → Inventory = One-to-One (1:1)

The product_id is used to connect the Product table with the Inventory table.

3. Applying Constraints

PRIMARY KEY

Uniquely identifies each record in a table.

FOREIGN KEY

Connects two tables.

UNIQUE

Prevents duplicate values.

NOT NULL

Does not allow an empty value.

CHECK

Ensures the value follows a given condition.

Insert Values

Insert values into **Seller Table**.

```
SQL> INSERT INTO Seller VALUES (1, 'Fresh Farm Organics', 'freshfarm@gmail.com');
1 row created.
SQL> INSERT INTO Seller VALUES (2, 'Green Valley Foods', 'greenvalley@gmail.com');
1 row created.
SQL> INSERT INTO Seller VALUES (3, 'Nature Basket', 'naturebasket@gmail.com');
1 row created.
SQL> INSERT INTO Seller VALUES (4, 'Healthy Harvest', 'healthyharvest@gmail.com');
1 row created.
SQL> COMMIT;
Commit complete.
```

Insert values into **Inventory Table**.

```
SQL> INSERT INTO Inventory VALUES (1, 101, 50);
1 row created.
SQL> INSERT INTO Inventory VALUES (2, 102, 10);
1 row created.
SQL> INSERT INTO Inventory VALUES (3, 103, 0);
1 row created.
SQL> INSERT INTO Inventory VALUES (4, 104, 25);
1 row created.
SQL> COMMIT;
Commit complete.
```

4. Reports

Report 1: Seller-wise Product Report

This report shows the seller name and the number of products supplied by each seller.

This query connects the Seller and Product tables and counts how many products are supplied by each seller.

```
SQL> SELECT * FROM Seller;

SELLER_ID SELLER_NAME                EMAIL
-----
1 Fresh Farm Organics             freshfarm@gmail.com
2 Green Valley Foods              greenvalley@gmail.com
3 Nature Basket                   naturebasket@gmail.com
4 Healthy Harvest                 healthyharvest@gmail.com
```

Report 2: Stock Availability Report

This report displays the product name, available stock, and stock status.

The stock quantity is checked to display whether the product is **Available, Low Stock, or Out of Stock**.

```
SQL> SELECT p.product_name, i.stock_quantity, CASE WHEN i.stock_quantity = 0 THEN 'Out of Stock' WHEN i.stock_quantity <= 10 THEN 'Low Stock' ELSE 'Available' END AS stock_status FROM Product p JOIN Inventory i ON p.product_id = i.product_id;
```

PRODUCT_NAME	STOCK_QUANTITY	STOCK_STATUS
Organic Apples	50	Available
Brown Rice	10	Low Stock
Carrots	0	Out of Stock
Bananas	25	Available

Report 3: Inventory Analysis

Total Number of Products

This counts the total number of products recorded in the inventory.

```
SQL> SELECT COUNT(*) AS total_products FROM Inventory;
```

TOTAL_PRODUCTS
4

Products Out of Stock

This finds the number of products that have zero stock.

```
SQL> SELECT COUNT(*) AS out_of_stock FROM Inventory WHERE stock_quantity = 0;
```

OUT_OF_STOCK
1

Highest Stocked Product

This identifies the product with the highest available stock.

```
SQL> SELECT p.product_name, i.stock_quantity FROM Product p JOIN Inventory i ON p.product_id = i.product_id WHERE i.stock_quantity = (SELECT MAX(stock_quantity) FROM Inventory);
```

PRODUCT_NAME	STOCK_QUANTITY
Organic Apples	50

Average Inventory Quantity

This calculates the average stock quantity of all products.

```
SQL> SELECT AVG(stock_quantity) AS average_inventory FROM Inventory;
```

AVERAGE_INVENTORY
21.25